DYNAMIC PROGRAMMING CONSTRAINT PROBLEMS

ปราการ อุณจักร
ภาควิชาวิทยาการคอมพิวเตอร์, คณะวิทยาศาสตร์, มหาวิทยาลัยเชียงใหม่

OUTLINE

- Dynamic Programming with Constraint Problems
- Pseudo-polynomial Time Algorithm
- Example: 0/1 Knapsack Problem
- Example: Partition Problem
- Practice Problem
  - Boss fight (C2P58)
  - Subset Sum (C2P59)
  - Fruit Merchant (C2E06)
  - [Extra] Mr. Robot (C2P83)

CONSTRAINT PROBLEM

- ปัญหาบางปัญหามีข้อจำกัด (Constraint) ที่คำตอบของเราไม่ควรละเมิด
  - คำตอบอาจเป็นค่าที่ดีที่สุด หรือว่าสามารถทำบางอย่างได้หรือไม่
- "จัดของกระเป๋าอย่างไรให้ได้มูลค่ามากที่สุดโดยที่น้ำหนักไม่เกินที่กระเป๋ารับได้"
- "มีเซตย่อยในเซตของตัวเลขที่ให้ที่บวกรวมกันได้ค่าที่กำหนดหรือไม่"

DYNAMIC PROGRAMMING

- Dynamic Programing (DP) แก้ปัญหาโดยหลีกเลี่ยงการแก้ปัญหาซ้ำ
  - Bottom-up เริ่มจากปัญหาย่อย แล้วนำผลมาประกอบรวมเป็นผลการแก้ปัญหาใหญ่
  - Top-down เริ่มจากปัญหาใหญ่ แต่จะมีการบันทึกผลการแก้ปัญหาย่อยไว้ (Memoization) ไม่ต้องทำซ้ำ
- ก่อนอื่น เราจะต้องคิดว่า จะแบ่งปัญหาเป็นปัญหาย่อยได้อย่างไร (Recursion และ Argument ใน Recursive Function)
- และใน DP จะต้องคิดด้วยว่า จะเก็บข้อมูลในรูปใด

❖ **Note**: ตัวอย่างในสไลด์นี้เริ่มนับจาก 1 แต่ในโค้ดมักจะเริ่มนับจาก 0 ทำให้บางส่วนต่างกัน

PSEUDO-POLYNOMIAL

- หลายปัญหาจะมี Algorithm แบบ Pseudo-polynomial Time มี Complexity เป็น Polynomial กับ **<u>ค่า</u>**ของ Input (ไม่ใช่ขนาด)
- ถ้า DP ของปัญหาเป็น Pseudo-polynomial แสดงว่าอาจมีมิติของข้อมูลที่เก็บไว้ เป็น Polynomial กับ**<u>ค่า</u>**ข้อมูล Input

KNAPSACK

- จัดของใส่กระเป๋าอย่างไร ให้ได้คุณค่า มากที่สุด
- มีกระเป๋า 1 ใบ มีของ n ชิ้น
- Input: W น้ำหนักที่กระเป๋ารองรับได้

  w_i น้ำหนักของสัมภาระชิ้นที่ i

  v_i คุณค่าของสัมภาระชิ้นที่ i
- **เป้าหมาย**: เลือกสัมภาระที่จะใส่กระเป๋าให้ได้มูลค่ารวม มากที่สุด โดยที่น้ำหนักรวมไม่เกินน้ำหนักที่ กระเป๋ารองรับได้

ตัวอย่าง

i	v_i	w_i
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

KNAPSACK (ต่อ)

- <u>ถ้า ของแต่ละชิ้น สามารถตัดแบ่งได้</u> → ใช้ Greedy Algorithm
- แต่ถ้าเป็น 0/1 Knapsack ไม่สามารถแบ่งได้
  - ถ้าจะเก็บใส่กระเป๋าต้องใส่ทั้งชิ้น
- จากตัวอย่าง W = 11
  - Optimal solution คือ เลือกชิ้นที่ 3 และ 4 ได้มูลค่า 40 หน่วย
- ทางเลือกตรงหน้าที่ดูดีที่สุด อาจไม่ใช่องค์ประกอบของ Optimal Solution

SOLVING 0/1 KNAPSACK – RECURSION

- ตัวเลือกสำหรับของแต่ละชิ้น คือ (1) ไม่นำใส่กระเป๋า หรือ (2) นำใส่กระเป๋า
  1. ไม่นำใส่กระเป๋า → ไปดูชิ้นถัดไป โดยที่น้ำหนัก / มูลค่าในกระเป๋าไม่เปลี่ยน
  2. นำใส่กระเป๋า → ไปดูชิ้นถัดไป โดยที่น้ำหนักที่รับได้ลดลง/มูลค่าเพิ่มขึ้นตามน้ำหนักของของชิ้นนั้น
- ใช้ KS(w, i) เป็นมูลค่ามากที่สุดเมื่อในสินค้าชิ้นที่ 1..i (พิจารณาตัวที่ i) ภายใต้ขอบเขตน้ำหนัก w มีตัวเลือก
- Base case: ถ้า i หรือ w เป็น 0 → return 0
- Recursive Case: ค่ามากที่สุดระหว่าง
  - ไม่ใช้ชิ้นที่ i → $KS(w, i) = KS(w, i - 1)$
  - ใช้ → $KS(w, i) = v_i + KS(w - w_i, i - 1)$

RECURSION PSEUDOCODE

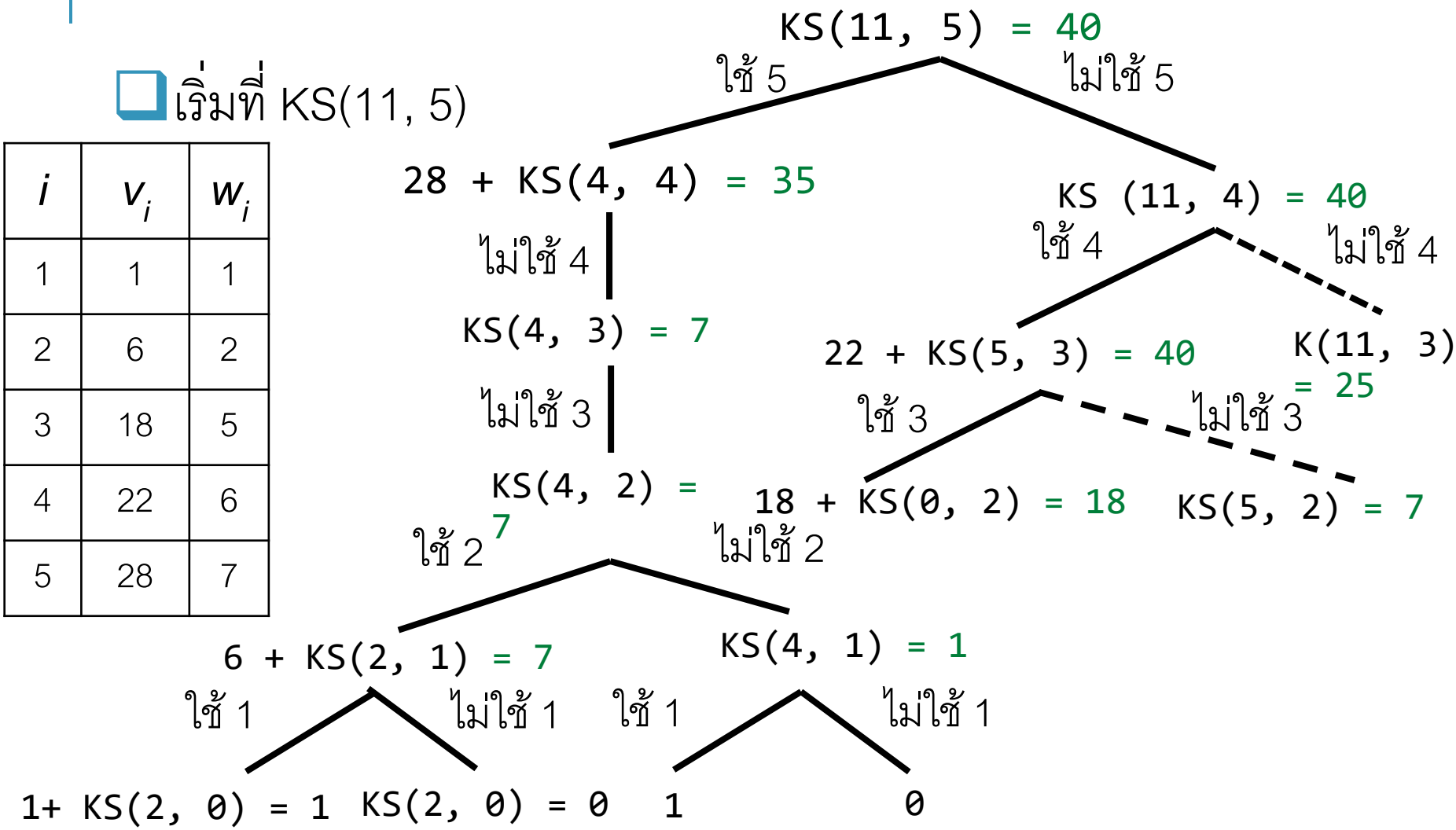
```
long knapsack(int w, int i) {
      if (i == 0 || w == 0) return 0;
      else if(w[i] > w) {
            return KnapSack(w, i - 1);
      } else {
            return max(KnapSack(w, i - 1)
                  , v[i] + KnapSack(w – w[i],
                                    i – 1,));
      }
}
```

- เรียก KnapSack(n, W) เพื่อหาคำตอบ
- Time complexity $O(2^n)$

RECURSION EXAMPLE

- เริ่มที่ KS(11, 5)

i	v_i	w_i
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7



DP FOR KNAPSACK – BOTTOM-UP

- รูป Recursive เป็น KS(w, i)
- สร้างตาราง ks[0..n][0..W] บรรจุ ค่า KS(w, i)
- คำตอบจะอยู่ที่ ks[n][W]

BOTTOM-UP PSEUDOCODE

```
void makeKS(int W, int n) {
   for(int i = 0; i <= n; i++){
      for(int j = 0; j <= W; j++){
         if(i == 0 || j == 0){
               ks[i][j] = 0;
         } else if (w[i] > k ) {
               ks[i][j] = ks[i-1][j];
         } else {
               ks[i][j] = max(ks[i-1][j]
                              , v[i] + ks[i-1][k – w[i]]);
         }
      }
   }
}
```

BOTTOM-UP EXAMPLE

i	v_i	w_i
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

	j = 0	1	2	3	4	5	6	7	8	9	10	11
i = 0												
1												
2												
3												
4												
5												

BOTTOM-UP EXAMPLE

	j = 0	1	2	3	4	5	6	7	8	9	10	11
i = 0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	1	1	1	1	1	1	1	1	1	1
2	0	1	6	7	7	7	7	7	7	7	7	7
3	0	1	6	7	7	18	19	24	25	25	25	25
4	0	1	6	7	7	18	22	23	28	29	29	40
5	0	1	6	7	7	18	22	28	29	34	35	40

ตัวอย่างโค้ด BOTTOM-UP

- knapsack_bu.cpp เริ่มนับจาก 0
- ที่ i = 0 (ของชิ้นแรก) ถ้าสามารถใส่ได้จะคืนค่า v[0] ถ้าไม่ได้จะคืน 0

```
int P[n][W+1];
// perform bottom up dynamic programing
for(int i = 0; i < n; i++){
    for(int j = 0; j <= W; j++){
        if(j == 0){ // Base case 1: no weight left
            P[i][j] = 0;
        } else {
            if(i == 0){ // Base case 2: first item
                if(itemArray[i].w > j){
                    P[i][j] = 0;    // can't fit
                } else { P[i][j] = itemArray[i].v; }
            } else {
                if(itemArray[i].w > j){
                    P[i][j] = P[i-1][j]; // can't fit
                } else {
                    // max between use or not use
                    P[i][j] = max(itemArray[i].v + P[i-1][j-itemArray[i].w]
                                ,P[i-1][j]);
                }
            }
        }
    }
}
cout << P[n-1][W] << endl;   // Answer
```

DP FOR KNAPSACK – TOP-DOWN

- คงการเรียกแบบ Recursion ไว้ โดยใช้ข้อมูลเพิ่มเติมเพื่อกันการทำซ้ำ – Array ks[0..n][0..W] เหมือนของ bottom-up
  - ks[i][w] แทนค่าที่ดีที่สุดเมื่อสามารถใช้ชิ้นที่ 1..i และมีน้ำหนักจำกัดที่ w
  - ในช่องที่ยังไม่ได้ทำการคำนวณ ให้ค่าเป็น -1 ไว้ก่อน เพื่อให้ง่ายต่อการตรวจสอบ
  - เพื่อให้ง่ายต่อการเขียนโค้ดอาจให้ค่าเริ่มต้นของบางช่องไว้เลย เช่น ks[i][0] = 0 และ ks[0][j] = 0
  - อาจไม่ได้คำนวณทุกช่อง
  - เริ่มการเรียกที่ `knapsack(n, W)`

TOP-DOWN PSEUDOCODE

```
long knapsack(int i, int w) {
        if (i == 0 || w == 0) {
                return 0;
        } else if(ks[i][w] != -1) return ks[i][w];
        else if(w[i] > w) {
                ks[i][w] = KnapSack(i - 1, w);
                return ks[i][w];
        } else {
                 ks[i][w] = max(KnapSack(i - 1, w)
                        , v[i] + KnapSack(i – 1
                                , w – w[i]));
                return ks[i][w];
        }
}
```

PRACTICE – TOP-DOWN

- ลองทำแบบ Top-down

i	v_i	w_i
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

	j = 0	1	2	3	4	5	6	7	8	9	10	11
i = 0	0	0	0	0	0	0	0	0	0	0	0	0
1	0											
2	0											
3	0											
4	0											
5	0											

PRACTICE – TOP-DOWN

- ลองทำแบบ Top-down (-1 คือช่องที่ไม่มีการคำนวณค่า)

	j = 0	1	2	3	4	5	6	7	8	9	10	11
i = 0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	-1	1	1	1	1	1	-1	-1	1	-1	1
2	0	-1	-1	-1	7	7	7	-1	-1	-1	-1	7
3	0	-1	-1	-1	7	18	-1	-1	-1	-1	-1	25
4	0	-1	-1	-1	7	-1	-1	-1	-1	-1	-1	40
5	0	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	40

ตัวอย่างโค้ด

- knapsack_td.cpp เริ่มนับจาก 0 เช่นกัน

```
long knapsackTopDown(long w, vector<ksItem> itemArray, int i, vector<vector<long>> &P){
    long value = 0;
    // base case, first item
    if (i == 0){
        // no previous item to consider, either use, or return 0
        value = 0;
        if(w >= itemArray[i].w){
            value = itemArray[i].v; // no Repeat!
        }
    } else { // Recursive case

        // 1st case: don't use i-th item
        if(P[i-1][w] == -1){ // check if the value has already been compute
            value = knapsackTopDown(w, itemArray, i-1, P); // if not, compute for the first time
        }
        else value = P[i-1][w];

        // 2nd case: use i-th item (if possible)
        if(w >= itemArray[i].w){
            long newValue;
            if(P[i-1][w - itemArray[i].w] == -1){
                newValue = itemArray[i].v + knapsackTopDown(w - itemArray[i].w, itemArray, i-1, P);
            }
            else newValue = itemArray[i].v + P[i-1][w - itemArray[i].w];
            if(newValue > value){
                value = newValue;
            }
        }
    }
    P[i][w] = value; // update DP Table
    return value;
}
```

KNAPSACK DP COMPLEXITIES

- Space/Time Complexity: O(nW)
- การที่มี W ใน Polynomial O() ทำให้การแก้ปัญหาแบบนี้เป็น Pseudo-polynomial Time Algorithm
  - ซึ่ง W เป็นขนาดหนึ่งของมิติในตารางด้วย
- ในปัญหาคล้าย ๆ กัน ก็อาจมีการใช้ค่า Input มาประกอบมิติของตารางได้
- กรณีของ Bottom-up ช่องที่คำนวณไป อาจไม่ได้ใช้ทุกช่องก็ได้
- ส่วนกรณีของ Top-down จะเห็นว่า มีช่องที่ไม่ประมวลผล (-1) อยู่จำนวนหนึ่ง อาจใช้ Hash Table เช่น Unsorted Map แทน Array ได้

PARTITION PROBLEM

- มีชุดตัวเลขจำนวนเต็ม $x_1, x_2, \ldots, x_n$ จงหาว่าสามารถแบ่งชุดตัวเลขนี้เป็นสองชุดย่อย ที่มีผลรวมเท่ากัน**<u>ได้หรือไม่</u>**
- ตัวอย่าง {2, 3, 2, 5, 4} สามารถแบ่งเป็น {2, 2, 4} และ {3, 5} ที่มีผลรวมเป็น 8 เท่ากันได้
- มีตัวเลือกที่ใด อย่างไรบ้าง
- รูป Recursive เป็นอย่างไร
- ถ้า Brute Force จะทำอย่างไร ได้ Complexity ในรูปใด

PARTITION PROBLEM

- $x_1 \ldots x_5 = \{2, 3, 2, 5, 4\}$

การแก้ PARTITION PROBLEM ด้วย DP

- ถ้า $\sum_{i=1..n} x_i = K$ ถ้าสามารถแบ่งชุดย่อยที่มีผลบวกรวมเท่ากันได้ ผลบวกที่ได้จะต้องเป็น $\lfloor K/2 \rfloor$
  - ถ้าสามารถหาได้ 1 ชุด ตัวเลขที่เหลือก็จะเป็นอีก 1 ชุด
  - กลายเป็น Constraint Problem หาชุดตัวเลขย่อยที่ทำให้ผลรวมเป็น $\lfloor K/2 \rfloor$
- Recursion
  - ให้ P(i, j) เป็น True ถ้า มีชุดตัวเลขย่อยใน $x_1, \ldots, x_i$ ที่มีผลรวมเป็น j
  - $P(i,j) = P(i-1,j)\ or\ P(i-1, j-x_i)$
  - Base Case: P(i, 0) = True
  - คำตอบจะอยู่ที่ $P(n, \lfloor K/2 \rfloor)$

PARTITION DP EXAMPLE

x = {2, 3, 2, 5, 4}, K/2 = 8 – ตาราง

	j = 0	1	2	3	4	5	6	7	8
i = 0									
1									
2									
3									
4									
5									

PARTITION DP EXAMPLE

x = {2, 3, 2, 5, 4}, K/2 = 8 – Base Case

	j = 0	1	2	3	4	5	6	7	8
i = 0	T								
1	T								
2	T								
3	T								
4	T								
5	T								

PARTITION DP EXAMPLE

x = {<u>2</u>, 3, 2, 5, 4}, K/2 = 8 – Loop i = 1, j = 1 to 8

	j = 0	1	2	3	4	5	6	7	8
i = 0	T								
1	T	F	T	F	F	F	F	F	F
2	T								
3	T								
4	T								
5	T								

PARTITION DP EXAMPLE

x = {2, <u>3</u>, 2, 5, 4}, K/2 = 8 – Loop i = 2, j = 1 to 8

	j = 0	1	2	3	4	5	6	7	8
i = 1	T	F	T	F	F	F	F	F	F
2	T	F	T	T	F	T	F	F	F
3	T								
4	T								
5	T								

PARTITION DP EXAMPLE

x = {2, 3, <u>2</u>, 5, 4}, K/2 = 8 – Loop i = 3, j = 1 to 8

	j = 0	1	2	3	4	5	6	7	8
i = 1	T	F	T	F	F	F	F	F	F
2	T	F	T	T	F	T	F	F	F
3	T	F	T	T	T	T	F	T	F
4	T								
5	T								

PARTITION DP EXAMPLE

x = {2, 3, 2, **<u>5</u>**, 4}, K/2 = 8 – Loop i = 4, j = 1 to 8

	j = 0	1	2	3	4	5	6	7	8
i = 1	T	F	T	F	F	F	F	F	F
2	T	F	T	T	F	T	F	F	F
3	T	F	T	T	T	T	F	T	F
4	T	F	T	T	T	T	F	T	T
5	T								

PARTITION DP EXAMPLE

x = {2, 3, 2, 5, <u>4</u>}, K/2 = 8 – Loop i = 5, j = 1 to 8

P(5, 8) = True!

	j = 0	1	2	3	4	5	6	7	8
i = 1	T	F	T	F	F	F	F	F	F
2	T	F	T	T	F	T	F	F	F
3	T	F	T	T	T	T	F	T	F
4	T	F	T	T	T	T	F	T	T
5	T	F	T	T	T	T	T	T	T

PARTITION DP PSEUDOCODE

```
void prepare_P(n, K_2){
   P[0][0] = true;
   for(int i = 0; i <= n; i++){
      P[i][0] = true;
      for(int j = 1; j <= K_2; j++){
         P[i][j] = P[i-1][j] || P[i-1][j-x[i];
      }
   }
}
// คำตอบจะอยู่ที่ P[n][K_2]
```

PARTITION DP EXAMPLE #2

- ลอง **x** = {1, 2, 3, 8}, K/2 = 7

	j = 0	1	2	3	4	5	6	7
i = 1								
2								
3								
4								

PRACTICE PROBLEMS

PRACTICE PROBLEMS

- ❑ C2P58: Boss Fight
- ❑ C2P59: Subset Sum
- ❑ C2E06: Fruit Merchant
- ❑ C2P83: Mr. Robot

PROBLEM – BOSS FIGHT (C2P58)

- ต้องการพกอาวุธไปสู้กับบอส ให้ได้พลังโจมตีรวมกันสูงที่สุด โดยที่น้ำหนักรวมกันไม่เกินที่กำหนด
  - ตัวอย่าง แบกน้ำหนักได้ 450

Weapon	Weight	Attack
1	80	60
2	100	120
3	180	150
4	350	300

- ปัญหานี้ คือปัญหาอะไร?

PROBLEM – SUBSET SUM (C2P59)

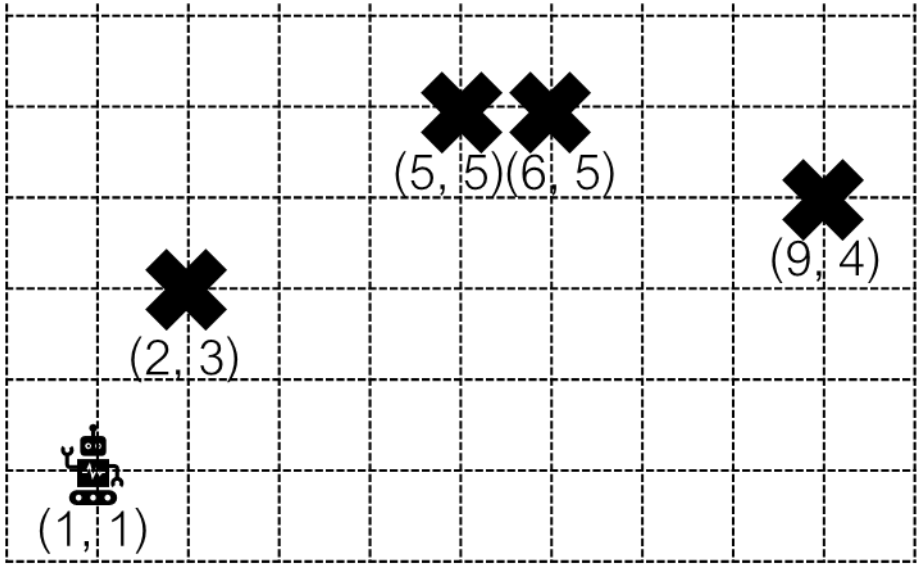
- กำหนด Array A และค่า M มาให้ ให้หาว่ามี Subset ของ A ที่มีผลรวมเท่ากับ M หรือไม่
- เช่น Array {4, 5, 0, 2, 9} และ M = 15
  - ได้ {4, 0, 2, 9} หรือ {4, 2, 9} ตอบ YES
- ข้อนี้ เหมือนหรือต่างจาก Knapsack / Partition อย่างไร
  - ตัวเลือกในแต่ละขั้น
  - รูป Recursive, ข้อมูลเพิ่มเติมที่จะเก็บไว้

PROBLEM – FRUIT MERCHANT (C2E06)

- เก็บลูกท้อวิเศษ ใส่ตะกร้า 2 ใบที่รับน้ำหนัก W_1 และ W_2 ภายใต้เวลา T ให้ได้ผลรวมมูลค่า v_i ,มากที่สุด
  - เก็บลูกท้อได้ทีละลูก ใช้เวลา t_i
  - ลูกท้อต้องใส่ตะกร้าใดตะกร้าหนึ่งโดยที่ลูกท้อในตะกร้าต้องมีน้ำหนัก w_i รวมกันไม่เกินที่ตะกร้านั้นจะรับน้ำหนักได้
  - ตัวอย่าง: $T = 9$, $W_1 = W_2 = 5$

i	v_i	t_i	w_i
1	2	3	2
2	4	4	5
3	3	5	3

PROBLEM – MR. ROBOT (C2P83)

- หาระยะทางน้อยที่สุด ที่หุ่นยนต์ทำความสะอาดจะต้องเดินทางจากจุดเริ่มต้น ไปยังจุดที่ต้องทำความสะอาด (จุดว้าว) ทุกจุดในห้อง แล้วกลับมาที่จุดเริ่มต้น
  - หุ่นยนต์เคลื่อนที่ได้ตามแนวแกน x และ y เท่านั้น จะวัดระยะทางระหว่างจุดสองจุดอย่างไร

ข้อมูลนำเข้า
10 10
1 1
4
2 3
5 5
9 4
6 5